МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федерального государственного бюджетного образовательного учреждения высшего образования

«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ Г.В.ПЛЕХАНОВА»

Техникум Пермского института (филиала)

Практическая работа №2

по дисциплине «Поддержка и тестирование программных модулей»

Руководитель _____ /Д.Б. Берестов/
«_17_» _марта_ 2026 г.

Исполнитель _____ /Д.Ю. Отинова/
«_17_» _марта_ 2026 г.

Пермь 2025 г.

Оглавление

ЗАДАНИЕ 4

ЧТО ТАКОЕ МОДУЛЬНОЕ ТЕСТИРОВАНИЕ И ЗАЧЕМ ОНО НУЖНО 4

ГЛАВА 1. СОЗДАНИЕ БИБЛИОТЕКИ КЛАССОВ 5

1.1 Запуск Visual Studio и создание проекта 5

1.2 Код класса Calculator 5

1.3 Сборка библиотеки 5

ГЛАВА 2. НАПИСАНИЕ МОДУЛЬНЫХ ТЕСТОВ 6

2.1 Создание проекта otinova_lib_tests 6

2.2 Подключение библиотеки к тестам 6

2.3 Создание тестов для арифметических операций 7

2.4 Тестирование исключений (деление на ноль) 8

2.5 Запуск тестов и результат 8

ГЛАВА 3. РАЗРАБОТКА ИНТЕРФЕЙСА WPF 9

3.1 Создание проекта otinova_pr2 9

3.2 Подключение библиотеки к интерфейсу 9

3.3 Создание кнопок и полей ввода 9

ГЛАВА 4. ЛОГИКА РАБОТЫ КАЛЬКУЛЯТОРА 11

4.1 Обработка нажатий кнопок 11

4.2 Вычисление выражений со скобками и степенью 11

4.3 Защита от неверного ввода 11

4.4 Обработка ошибок (деление на ноль) 11
ГЛАВА 5. ТЕСТИРОВАНИЕ КАЛЬКУЛЯТОРА 13
5.1 Проверка основных операций 13
5.2 Проверка сложных выражений 13
5.3 Проверка защиты от дурака 13
ЗАКЛЮЧЕНИЕ 15

ЗАДАНИЕ

Разработать программу «Калькулятор» на WPF, которая выполняет операции сложения, вычитания, умножения, деления и возведения в степень. Основную логику вынести в отдельную библиотеку классов. Для библиотеки написать модульные тесты. Результат работы выложить на GitHub, отправить ссылку преподавателю, оформить сопроводительную записку с иллюстрациями.

ЧТО ТАКОЕ МОДУЛЬНОЕ ТЕСТИРОВАНИЕ И ЗАЧЕМ ОНО НУЖНО

Модульные тесты — это небольшие программы, которые проверяют отдельные части кода. Например, есть метод, который складывает два числа. Тест подаёт на вход 2 и 3 и ожидает на выходе 5. Если метод возвращает 5 — тест проходит, если нет — падает.

Зачем это нужно? Во-первых, тесты помогают быстро находить ошибки. Во-вторых, если в будущем кто-то захочет изменить код, тесты сразу покажут, не сломалось ли что-то. В этой работе тесты написаны для каждого метода библиотеки: сложение, вычитание, умножение, деление, степень, а также проверка деления на ноль.

ГЛАВА 1. СОЗДАНИЕ БИБЛИОТЕКИ КЛАССОВ

1.1 Запуск Visual Studio и создание проекта

Я открыла Visual Studio и нажала «Создать новый проект». Выбрала шаблон «Библиотека классов». Назвала проект otinova_lib, а решение Calculator_PR2_Otinova. Версия .NET — 8.0.

Выбор нужного проекта можно увидеть на рисунке 1.1.

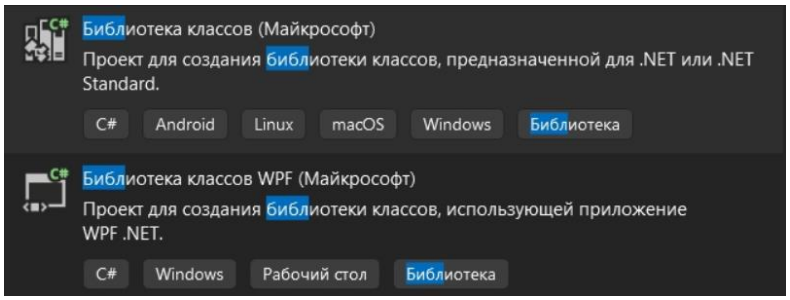

Рисунок 1.1 – **Создание проекта библиотеки**

1.2 Код класса Calculator

В проекте был создан файл Class1.cs. Я переименовала его в Calculator.cs и написала код. В классе пять методов: Add, Subtract, Multiply, Divide, Power. В методе Divide добавила проверку: если второй аргумент равен нулю — выбрасывается исключение с сообщением.

1.3 Сборка библиотеки

После написания кода я нажала «Построить решение» (Ctrl+Shift+B), чтобы убедиться, что ошибок нет. Всё собралось успешно.

ГЛАВА 2. НАПИСАНИЕ МОДУЛЬНЫХ ТЕСТОВ

2.1 Создание проекта otinova_lib_tests

Дальше нужно было добавить проект для тестов. Я нажала правой кнопкой на решении, выбрала «Добавить» => «Создать проект». В поиске ввела «test» и выбрала «Тестовый проект MSTest (Майкрософт)». Назвала его otinova_lib_tests.

Выбор нужного проекта можно увидеть на рисунке 2.1.

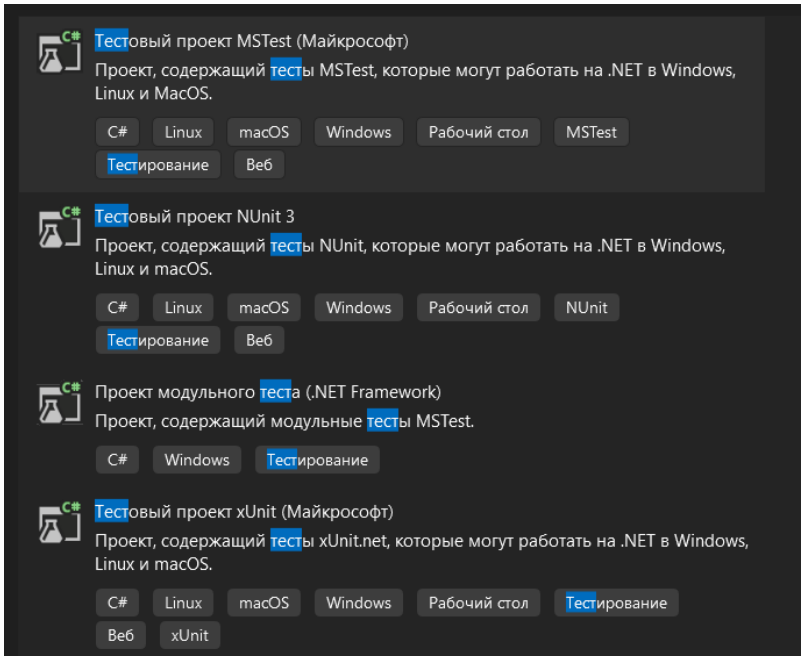

Рисунок 2.1 – **Создание проекта тестов**

2.2 Подключение библиотеки к тестам

Чтобы тесты видели код из библиотеки, я добавила ссылку. В проекте otinova_lib_tests нажала правой кнопкой на «Зависимости» => «Добавить ссылку на проект» и выбрала otinova_lib.

Окно зависимостей можно увидеть на рисунке 2.2.

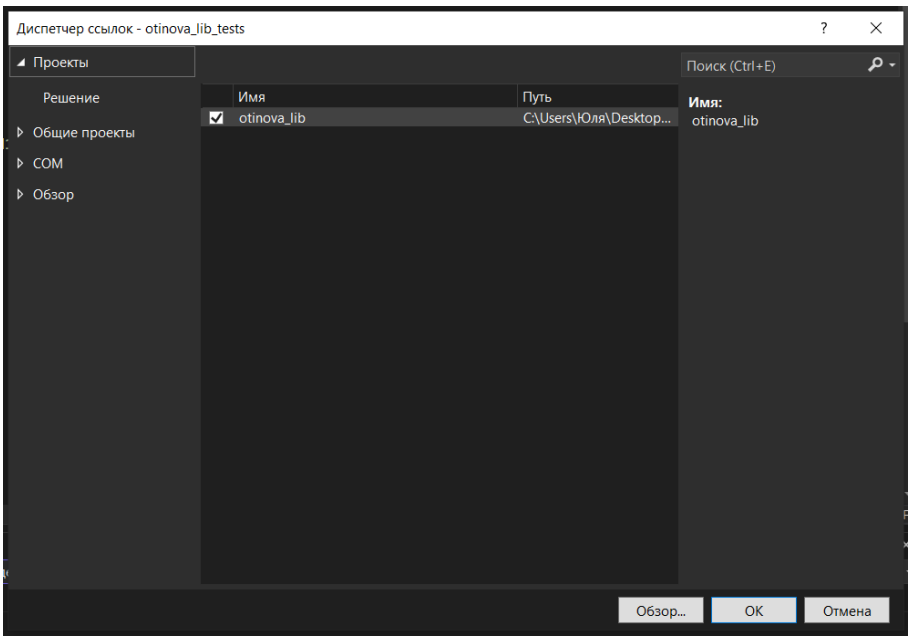

Рисунок 2.2 – **Подключение библиотеки**

2.3 Создание тестов для арифметических операций

В тестовом проекте был файл UnitTest1.cs. Я переименовала его в CalculatorTests.cs. Внутри написала тесты для каждого метода:

• проверка сложения (2 + 3 = 5)

• проверка вычитания (10 – 4 = 6)

• проверка умножения (6 * 7 = 42)

• проверка деления (15 / 3 = 5)

• проверка степени (2 ^ 3 = 8)

Каждый тест устроен по принципу: подготовка данных, выполнение, проверка результата.

2.4 Тестирование исключений (деление на ноль)

Отдельно проверила, что при делении на ноль программа выбрасывает исключение DivideByZeroException. Для этого написала тест, который ожидает именно такую ошибку.

2.5 Запуск тестов и результат

Я открыла «Обозреватель тестов» (Тест => Обозреватель тестов) и нажала «Запустить все». Все 6 тестов загорелись зелёным, что можно увидеть на рисунке 2.5. Это значит, что библиотека работает правильно.

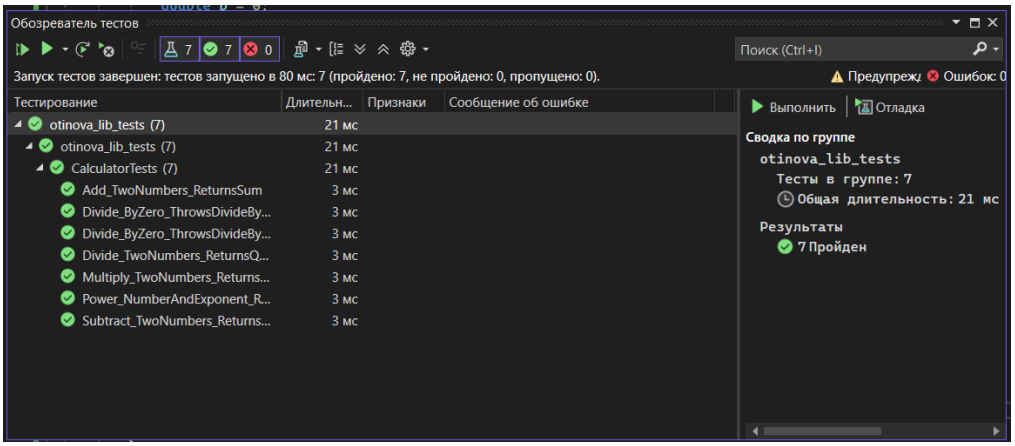

Рисунок 2.5 – **Результаты тестов**

ГЛАВА 3. РАЗРАБОТКА ИНТЕРФЕЙСА WPF

3.1 Создание проекта otinova_pr2

Снова нажала правой кнопкой на решении, выбрала «Добавить» => «Создать проект». На этот раз выбрала «Приложение WPF (Майкрософт)». Назвала проект otinova_pr2. Версия .NET — тоже 8.0, чтобы совпадала с библиотекой.

3.2 Подключение библиотеки к интерфейсу

В проекте otinova_pr2 добавила ссылку на otinova_lib так же, как в тестах: правой кнопкой на «Зависимости» => «Добавить ссылку на проект». Без этого программа не увидит методы калькулятора.

3.3 Создание кнопок и полей ввода

В файле MainWindow.xaml я разместила:

• два текстовых поля: сверху маленькое — для выражения, снизу большое — для ввода чисел и результата;

• кнопки для цифр, операций, скобок, очистки и равно;

• добавила стили, чтобы кнопки красиво подсвечивались при наведении.

Вид самого калькулятора можно увидеть на рисунке 3.3.

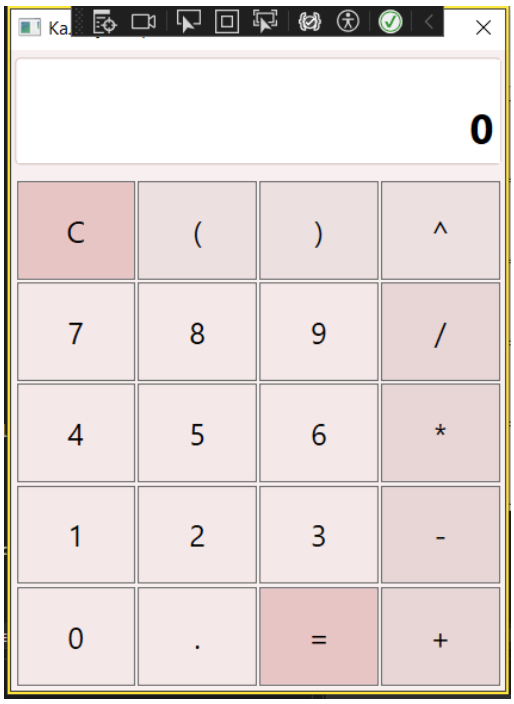

Рисунок 3.3 – **Интерфейс калькулятора**

ГЛАВА 4. ЛОГИКА РАБОТЫ КАЛЬКУЛЯТОРА

4.1 Обработка нажатий кнопок

В файле MainWindow.xaml.cs я написала обработчики событий. При нажатии на цифру или оператор символ добавляется в поле ввода. При нажатии на «C» всё очищается. При нажатии на «=» начинается вычисление.

4.2 Вычисление выражений со скобками и степенью

Самая сложная часть — разбор выражения. Я написала метод, который сначала находит все скобки и вычисляет их содержимое, потом обрабатывает степень, потом умножение и деление, потом сложение и вычитание. Все арифметические операции вызываются из библиотеки otinova_lib.

4.3 Защита от неверного ввода

Чтобы пользователь случайно не сломал программу, я добавила защиту:

• нельзя поставить две точки в одном числе (например, 1..5);

• нельзя поставить два оператора подряд (++);

• нельзя поставить оператор после точки без цифры;

• если скобки не закрыты — при нажатии «=» выдаётся ошибка

4.4 Обработка ошибок (деление на ноль)

Если пользователь делит на ноль, программа ловит исключение DivideByZeroException и показывает понятное сообщение, а не просто вылетает.

Как выглядит ошибка, можно увидеть на рисунке 4.4.

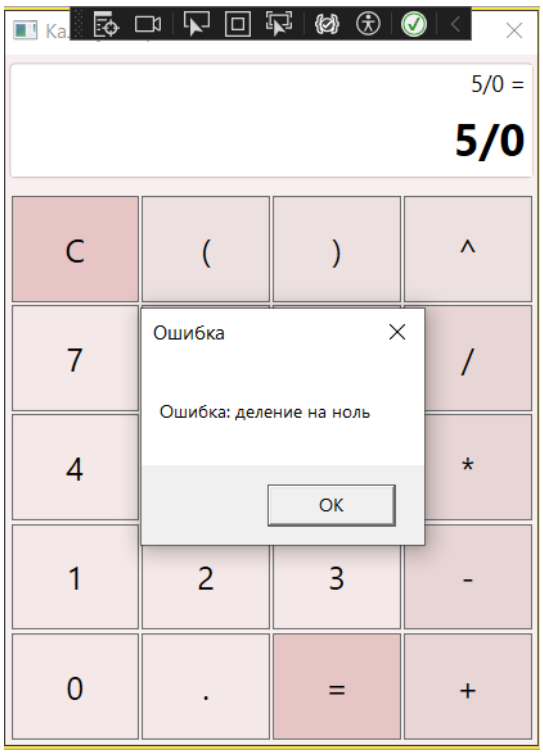

Рисунок 4.4 – **Ошибка, деление на ноль**

ГЛАВА 5. ТЕСТИРОВАНИЕ КАЛЬКУЛЯТОРА

5.1 Проверка основных операций

Я вручную проверила, что калькулятор правильно считает:

• 2 + 3 = 5

• 5 – 2 = 3

• 4 * 3 = 12

• 9 / 3 = 3

• 2 ^ 3 = 8

Всё работает.

5.2 Проверка сложных выражений

Проверила со скобками и степенью:

• (2 + 3) * 4 = 20

• 2 ^ (1 + 2) = 8

• (2 + 3) ^ 2 = 25

Всё считается правильно.

5.3 Проверка защиты от дурака

Проверила, что нельзя ввести 1..5 или 2++3. Калькулятор такие символы просто не добавляет. При незакрытых скобках выдаёт ошибку.

Как выглядит ошибка при не закрытых скобках, можно увидеть на рисунке 5.3.

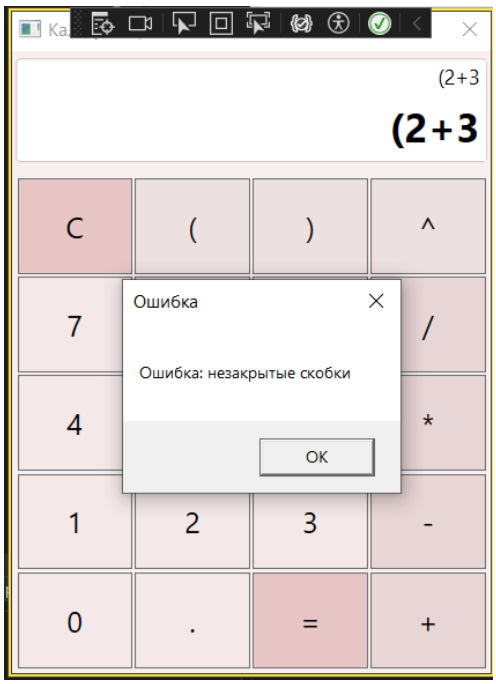

Рисунок 5.3 – **Ошибка, незакрытые скобки**

ЗАКЛЮЧЕНИЕ

В ходе работы я сделала полноценный калькулятор на WPF. Вся логика вынесена в отдельную библиотеку, для неё написаны модульные тесты, которые успешно проходят. Интерфейс получился удобным, с защитой от неверного ввода и понятными сообщениями об ошибках.

Я научилась:

• создавать много проектов в одном решении;

• подключать библиотеки к WPF и тестам;

• писать модульные тесты и понимать их результаты;

• разбирать математические выражения со скобками и степенью;

• защищать программу от некорректного ввода.

Работа выложена на GitHub, ссылка отправлена преподавателю.